Name: Ronit Sahoo
Class: D20C
Roll No: 29
Batch: B

BLOCKCHAIN EXPERIMENT NO: 3

Aim - Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory:

1. Blockchain Overview

Blockchain is a distributed and decentralized ledger that stores information in a series of linked blocks. Each block contains:

- Transaction data
- Timestamp
- Previous block's hash
- Its own unique hash (digital fingerprint)

Once data is recorded in a blockchain it becomes immutable, because altering one block would require recalculating the hash of every subsequent block.

2. Mining

Mining is the process of:

- Collecting pending transactions into a block.
- Performing a computational puzzle (Proof-of-Work) to find a valid hash.
- Adding the new block to the blockchain.
- Broadcasting it to all connected peers.

Miners are rewarded with cryptocurrency for successfully mining a block.

3. Multi-Node Blockchain Network

In this experiment three independent blockchain nodes are simulated on ports 5001, 5002 and 5003. Each node runs on a separate port, maintains its own copy of the blockchain, and can connect with peers to share and validate blocks.

4. Consensus Mechanism

The Longest Chain Rule is used as the consensus mechanism. If multiple versions of the chain exist across the network, the longest valid chain is chosen. This ensures that all nodes eventually agree on a single transaction history.

5. Transactions and Mining Reward

Each transaction contains a sender, a receiver and an amount. When a block is mined, the pending transactions are added to the block and a reward transaction is automatically appended to pay the miner for the computational work performed.

6. Chain Replacement

When the /replace_chain endpoint is called, the node requests the chains held by its peers. If it finds a chain that is both longer and valid, it replaces its own copy with that chain. This is the mechanism that keeps the blockchain consistent across all nodes.

Software Requirements:

- Python 3.x

- Flask (web framework for the node API)
- Requests library (for peer-to-peer communication)
- Postman (for sending API requests)
- Web browser (for GET endpoint verification)

Procedure:

1. Write the Blockchain class implementing block creation, Proof-of-Work, hashing, chain validation, transactions, node registration and chain replacement.
2. Expose the functionality through Flask endpoints: /mine_block, /get_chain, /is_valid, /add_transaction, /connect_node and /replace_chain.
3. Run three instances of the program on ports 5001, 5002 and 5003 to simulate three independent nodes.
4. Connect the nodes to each other by sending a POST request to /connect_node on each node with the addresses of the other two.
5. Add three transactions to node 5001 using POST requests to /add_transaction.
6. Mine blocks on node 5001 using /mine_block so the pending transactions are included and a mining reward is issued.
7. Inspect the chain on each node using /get_chain to observe the divergence between nodes.
8. Call /replace_chain on nodes 5002 and 5003 to demonstrate consensus using the Longest Chain Rule.
9. Validate the chain on all three nodes using /is_valid.

Code:

```
import datetime
import hashlib
import json
from uuid import uuid4
from urllib.parse import urlparse

import requests
from flask import Flask, jsonify, request



class Blockchain:

    def __init__(self):
        self.chain = []
        self.transactions = []
        self.create_block(proof=1,
previous_hash='0')
        self.nodes = set()

    def create_block(self, proof,
previous_hash):
        block = {'index': len(self.chain)
+ 1,
                 'timestamp':
str(datetime.datetime.now()),
                 'proof': proof,
                 'previous_hash':
previous_hash,
                 'transactions':
self.transactions}
        self.transactions = []
        self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

    def proof_of_work(self,
previous_proof):
        new_proof = 1
        while True:
            hash_operation =
hashlib.sha256(
                str(new_proof ** 2 -
previous_proof **
2).encode()).hexdigest()
            if
hash_operation.startswith('0000'):
                return new_proof
            new_proof += 1

    def hash(self, block):
        encoded_block = json.dumps(block,
sort_keys=True).encode()
        return
hashlib.sha256(encoded_block).hexdigest()
```

```
    def is_chain_valid(self, chain):
        if not chain:
            return False
        previous_block = chain[0]
        block_index = 1
        while block_index < len(chain):
            block = chain[block_index]
            if block['previous_hash'] !=
self.hash(previous_block):
                return False
            previous_proof =
previous_block['proof']
            proof = block['proof']
            hash_operation =
hashlib.sha256(
                str(proof ** 2 -
previous_proof **
2).encode()).hexdigest()
            if not
hash_operation.startswith('0000'):
                return False
            previous_block = block
            block_index += 1
        return True

    def add_transaction(self, sender,
receiver, amount):
        transaction = {'sender': sender,
'receiver': receiver, 'amount': amount}

self.transactions.append(transaction)
        previous_block =
self.get_previous_block()
        return previous_block['index'] +
1


    def add_node(self, address):
        parsed_url = urlparse(address)
        if parsed_url.netloc:

self.nodes.add(parsed_url.netloc)


    def replace_chain(self):
        network = self.nodes
        longest_chain = None
        max_length = len(self.chain)
        for node in network:
            try:
                response =
requests.get(f'http://{node}/get_chain',
timeout=5)
                if response.status_code
== 200:
                    data =
response.json()
                    length =
data['length']
                    chain = data['chain']
                    if length >
max_length and
self.is_chain_valid(chain):
                        max_length =
length
                        longest_chain =
chain
            except
requests.RequestException:
                continue
        if longest_chain is not None:
            self.chain = longest_chain
            return True
        return False



app = Flask(__name__)
node_address = str(uuid4()).replace('-',
'')
blockchain = Blockchain()



@app.route('/mine_block',
methods=['GET'])
def mine_block():
    previous_block =
blockchain.get_previous_block()
    previous_proof =
previous_block['proof']
    proof =
blockchain.proof_of_work(previous_proof)
    previous_hash =
blockchain.hash(previous_block)

blockchain.add_transaction(sender=node_ad
dress, receiver='Abcde', amount=1)
    block =
blockchain.create_block(proof,
previous_hash)
    response = {'message':
'Congratulations, you just mined a
block!',
                'index': block['index'],
                'timestamp':
block['timestamp'],
                'proof': block['proof'],
                'previous_hash':
block['previous_hash'],
                'transactions':
block['transactions']}
    return jsonify(response), 200



@app.route('/get_chain', methods=['GET'])
def get_chain():
    response = {'chain':
blockchain.chain, 'length':
len(blockchain.chain)}
    return jsonify(response), 200



@app.route('/is_valid', methods=['GET'])
def is_valid():
```

```
    valid =
blockchain.is_chain_valid(blockchain.chai
n)
    if valid:
        response = {'message': 'All good.
The Blockchain is valid.'}
    else:
        response = {'message': 'Houston,
we have a problem. The Blockchain is not
valid.'}
    return jsonify(response), 200



@app.route('/add_transaction',
methods=['POST'])
def add_transaction():
    transaction_data = request.get_json()
    if not transaction_data:
        return jsonify({'message':
'Request body must contain JSON'}), 400
    required_keys = ['sender',
'receiver', 'amount']
    if not all(key in transaction_data
for key in required_keys):
        return jsonify({'message': 'Some
elements of the transaction are
missing'}), 400
    index =
blockchain.add_transaction(transaction_da
ta['sender'],

transaction_data['receiver'],

transaction_data['amount'])
    response = {'message': f'This
transaction will be added to Block
{index}'}
    return jsonify(response), 201



@app.route('/connect_node',
methods=['POST'])
def connect_node():
    data = request.get_json()
    if not data:
        return jsonify({'message':
'Request body must contain JSON'}), 400
    nodes = data.get('nodes')
    if not nodes:
        return jsonify({'message': 'No
node'}), 400
    for node in nodes:
        blockchain.add_node(node)
    response = {'message': 'All the nodes
are now connected. The Hadcoin Blockchain
now '
                           'contains the
following nodes:',
                'total_nodes':
list(blockchain.nodes)}
    return jsonify(response), 201



@app.route('/replace_chain',
methods=['GET'])
def replace_chain():
    is_chain_replaced =
blockchain.replace_chain()
    if is_chain_replaced:
        response = {'message': 'The nodes
had different chains so the chain was
replaced '
                                'by the
longest one.',
                    'new_chain':
blockchain.chain}
    else:
        response = {'message': 'All good.
The chain is the largest one.',
                    'actual_chain':
blockchain.chain}
    return jsonify(response), 200



if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5001,
debug=False)
```

Output:

1. Connecting the Nodes Using Postman

Each of the three nodes was registered with the other two by sending a POST request to /connect_node containing the peer addresses. All three requests returned 201 CREATED with the list of connected nodes.

Output:

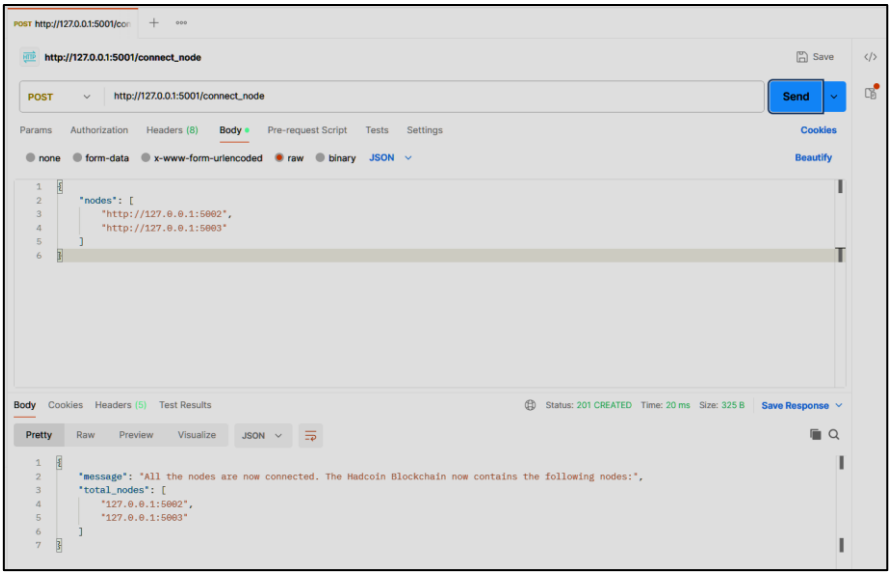


Fig 1: Node 5001 connected to peers 5002 and 5003.

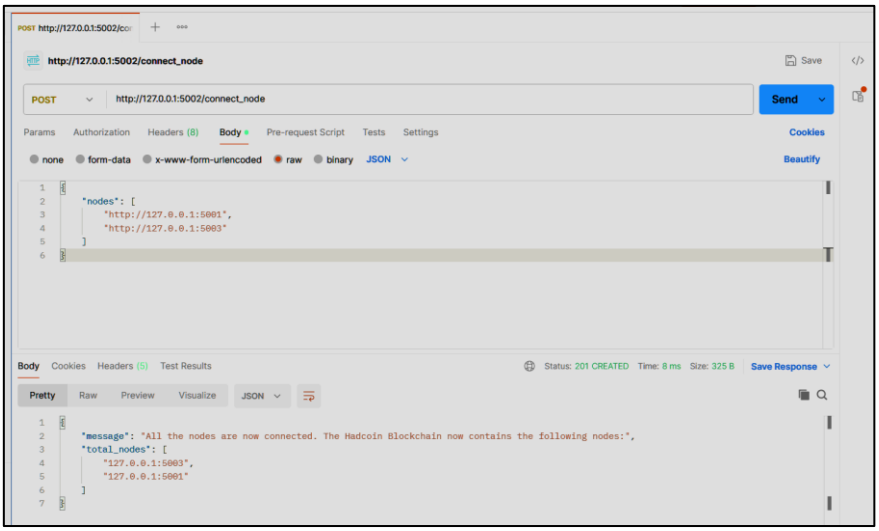


Fig 2: Node 5002 connected to peers 5001 and 5003.

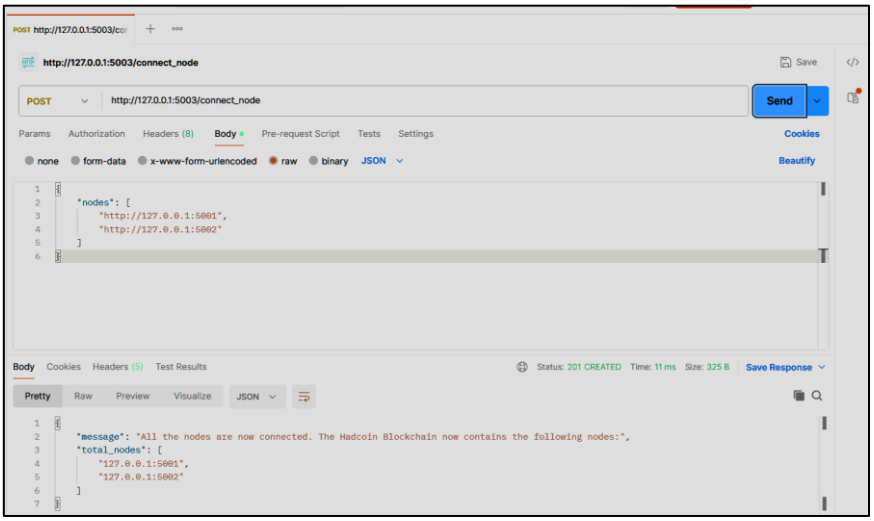


Fig 3: Node 5003 connected to peers 5001 and 5002.

2. Adding Three Transactions

Three transactions were submitted to node 5001 through POST requests to /add_transaction, each sending an amount from sender "a" to receiver "b". Each response confirmed that the transaction would be added to Block 2.

Output:

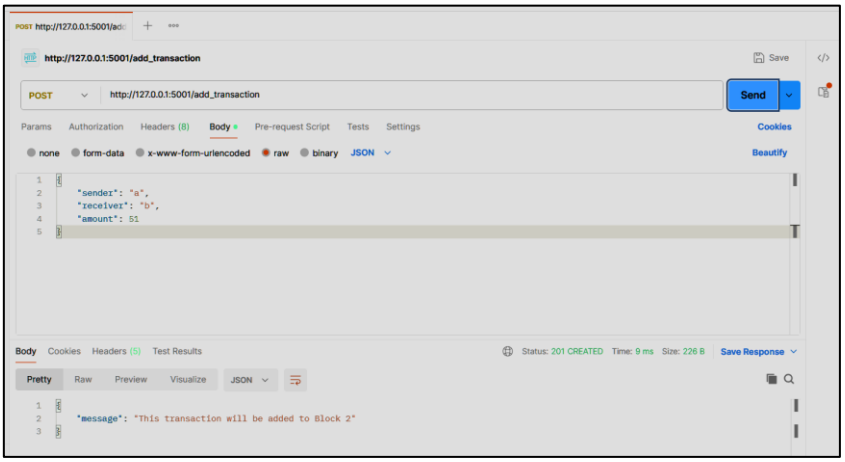


Fig 4: First transaction of amount 51 added to the pending list.

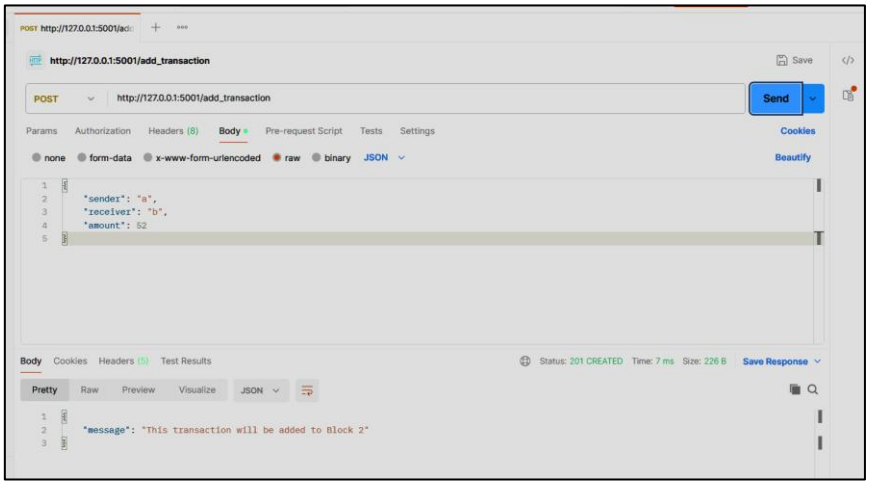


Fig 5: Second transaction of amount 52 added to the pending list.

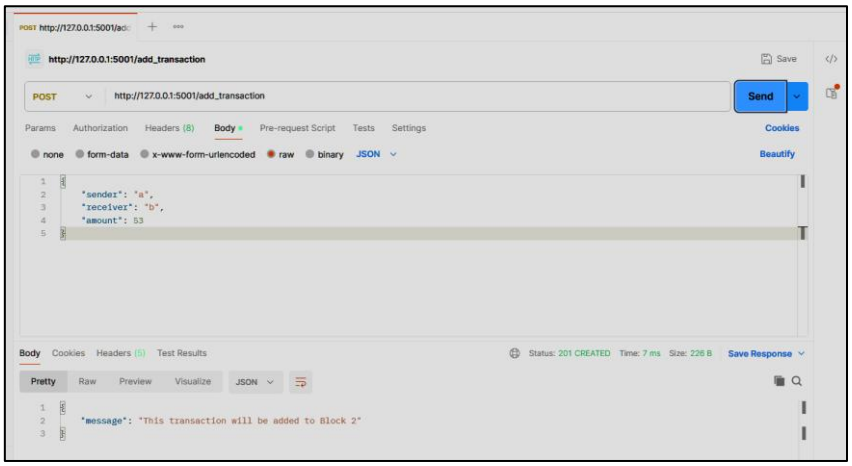


Fig 6: Third transaction of amount 53 added to the pending list.

3. Mining the Blocks

Calling /mine_block on node 5001 ran the Proof-of-Work algorithm until a hash starting with 0000 was found. Block 2 was mined with proof 533 and contained all three pending transactions plus the automatic mining reward of 1 unit paid to "Abcde".

Output:

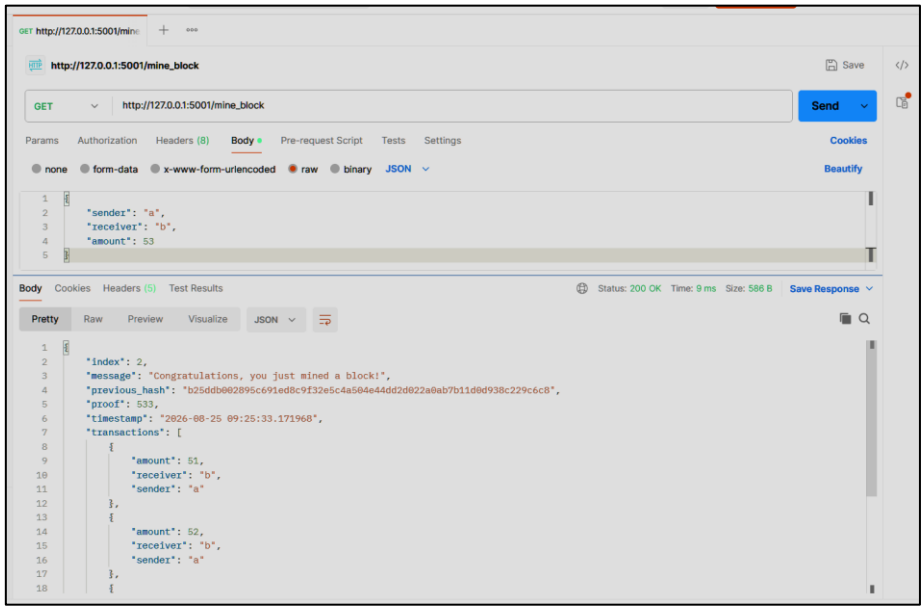


Fig 7: Block 2 mined with proof 533, containing the three transactions and the reward.

A second block was then mined. Because the pending transaction list had been cleared by the previous block, Block 3 contains only the mining reward transaction.

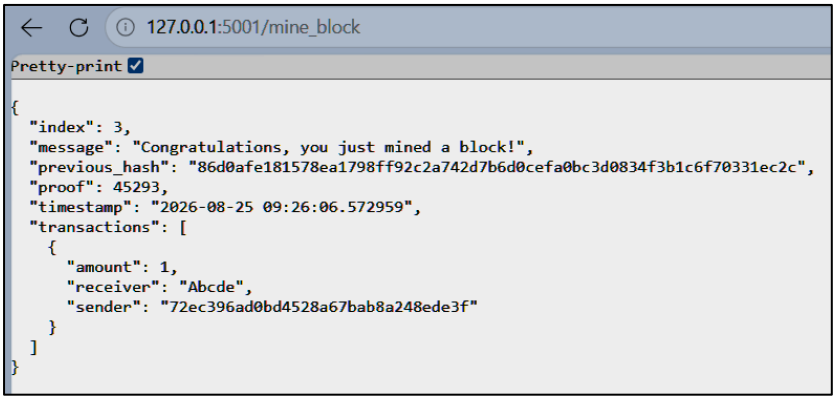


Fig 8: Block 3 mined with proof 45293, containing only the mining reward.

4. Checking the Chains Before Consensus

The chain held by each node was inspected using /get_chain. Node 5001 held three blocks, while nodes 5002 and 5003 still held only their genesis block, showing that the network had diverged because mining had taken place on a single node.

Output:

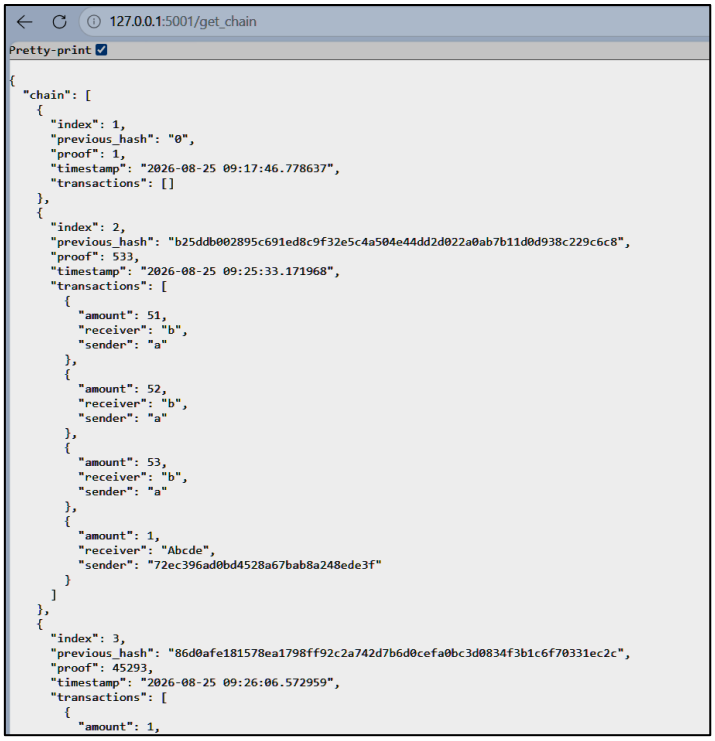


Fig 9: Node 5001 chain viewed in the browser, showing all three blocks and their transactions.

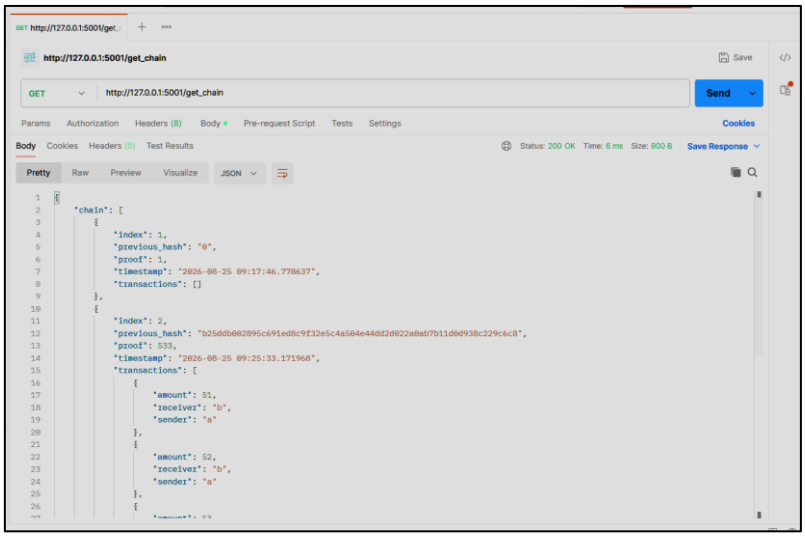


Fig 10: The same chain retrieved from node 5001 through Postman (200 OK, 900 B).

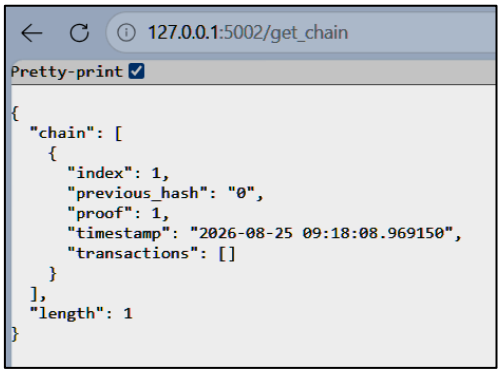


Fig 11: Node 5002 holds only the genesis block, with length 1.



Fig 12: Node 5003 also holds only the genesis block, with length 1.

5. Demonstrating Consensus

The /replace_chain endpoint was called on nodes 5002 and 5003. Each node requested the chains held by its peers, found the longer valid chain on node 5001, and adopted it. Both nodes then held the identical three-block chain.

Output:

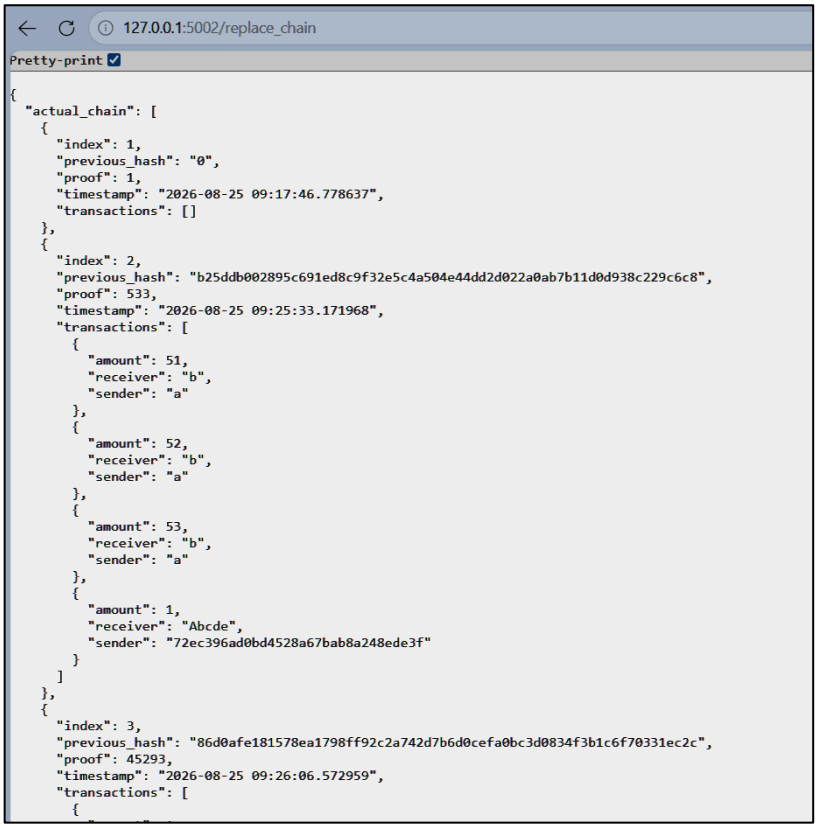


Fig 13: Node 5002 after chain replacement, now holding the full three-block chain.

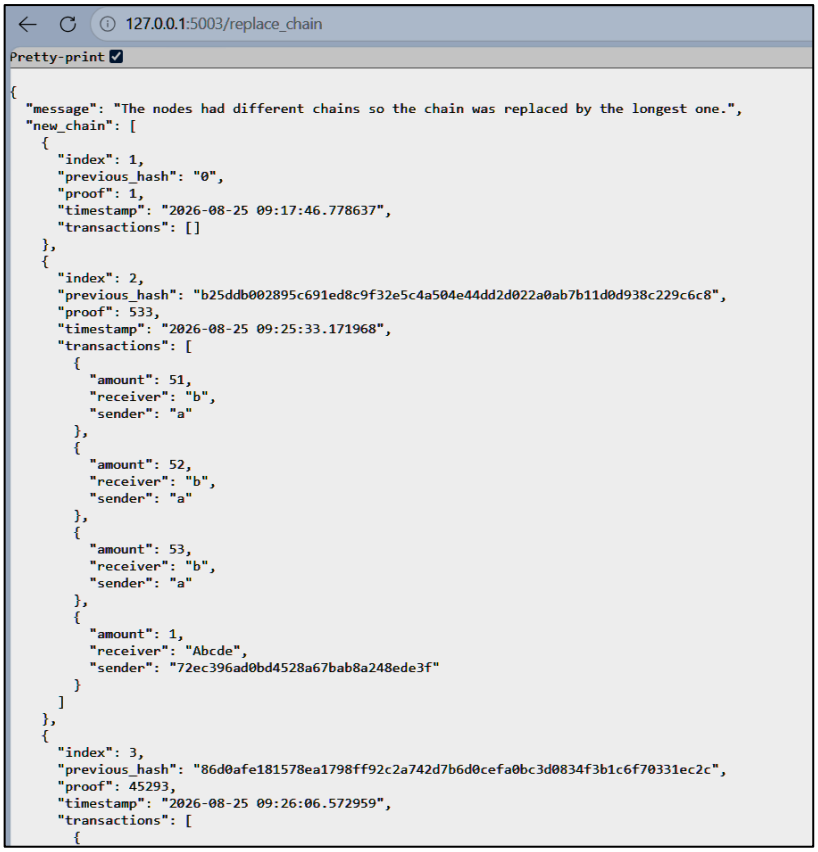


Fig 14: Node 5003 reporting that the chain was replaced by the longest one.

6. Validating All Three Nodes

Finally /is_valid was called on each node. All three confirmed that their chain satisfied both the hash-linkage check and the Proof-of-Work difficulty condition.

Output:



Fig 15: Validation results from node 5001 (left), node 5002 (centre) and node 5003 (right), all reporting a valid blockchain.

Summary of the Mined Chain:

Block	Proof	Previous Hash (truncated)	Transactions	Timestamp
1 (Genesis)	1	0	None	09:17:46.778637
2	533	b25ddb002895c691...938c229c6c8	3 transfers (51, 52, 53) + reward of 1	09:25:33.171968
3	45293	86d0afe181578ea1...6f70331ec2c	Mining reward of 1 only	09:26:06.572959

Observations:

1. **Proof-of-Work difficulty is visible in the proof values.** Block 2 required a proof of 533, while Block 3 required 45293. Both satisfy the same condition of producing a hash beginning with 0000, but the number of nonce values that had to be tested differed by almost two orders of magnitude. This illustrates that mining time is probabilistic rather than fixed.
2. **The pending transaction pool is cleared once a block is mined.** Block 2 contained all three transfers plus the reward, but Block 3 contained only the reward. This confirms that create_block() empties self.transactions after including them, so a transaction can never be recorded twice.
3. **The mining reward is generated by the network, not sent by a user.** Each mined block contains a transaction of amount 1 to "Abcde" from the node address 72ec396ad0bd4528a67bab8a248ede3f. This transaction was never submitted through /add_transaction; it is created automatically inside mine_block(), which is how new currency enters circulation.
4. **Connecting nodes does not synchronize them.** All three nodes were connected before mining, yet nodes 5002 and 5003 still held only their genesis block afterwards. Registration only makes peers aware of each other; synchronization happens exclusively when /replace_chain is called.
5. **The Longest Chain Rule resolved the divergence correctly.** Nodes 5002 and 5003 each held a chain of length 1 against node 5001's length 3. On calling /replace_chain both adopted the longer chain, and node 5003 explicitly reported that the chain had been replaced. The three ledgers became identical without any central coordinator.
6. **Replacement requires validity as well as length.** The replace_chain() method checks is_chain_valid(chain) before accepting a longer chain, so a node cannot be tricked into adopting a longer but tampered chain. This is what makes the consensus rule safe rather than merely majority-based.
7. **The genesis timestamps differ across nodes.** Node 5001 created its genesis block at 09:17:46, node 5002 at 09:18:08 and node 5003 at 09:18:31, reflecting the order in which the three processes were started. After consensus all three shared the 09:17:46 genesis block from node 5001, confirming that the entire chain was replaced rather than merged.

Conclusion:

In this experiment a cryptocurrency was developed using Python and Flask, and mining was performed in a simulated three-node blockchain network running on ports 5001, 5002 and 5003. Three transactions were submitted to node 5001 and mined into Block 2 using Proof-of-Work with a difficulty of four leading zeros, along with an automatic reward transaction paying the miner.

Before consensus, node 5001 held a chain of three blocks while nodes 5002 and 5003 held only their genesis blocks, demonstrating how independent nodes diverge when mining occurs on a single node. Calling /replace_chain on the two lagging nodes caused both to adopt the longer valid chain under the Longest Chain Rule, after which all three ledgers were identical and all three reported a valid blockchain through /is_valid.

The experiment therefore demonstrated the key blockchain concepts of Proof-of-Work mining, automatic reward issuance, decentralized consensus and network synchronization, and showed that chain replacement depends on validity as well as length, which is what prevents a longer but tampered chain from being accepted.